Docker:

1. Write a dockerfile. How to build it & how to run the image - cmds

Dockerfile:

```
# Use official Python image as base
FROM python:3.9
# Set the working directory inside the container
WORKDIR /app
# Copy the application files to the container
COPY . .
# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt
# Expose the application port
EXPOSE 5000
# Define the command to run the application
CMD ["python", "app.py"]
```

Steps to Build and Run the Docker Image

1. Build the Docker Image:
`docker build -t my-python-app .`

2. Run the Docker Container: `docker run -d -p 5000:5000 --name my-container my-python-app`

3. Verify the Running Container: `docker ps`

4. Check Logs: `docker logs -f my-container`

5. Stop the Container: `docker stop my-container`

6. Remove the Container: `docker rm my-container`

7. Remove the Image (if needed): `docker rmi my-python-app`

2. Scenario where docker image is 1GB size, how do we reduce the size of image?
Ans. Multi stage docker

3. When a container gets crashed and we want to access the data even after crash?
Ans. We can create volumes.

4. What strategy must be followed to generate secure or smaller Docker image?
Alpine Base, single stage, multi-stage docker.
Ans: multi-stage docker.

5. Pick the statement that are valid in Docker.
Argument. ARG used to define build time variable or runtime variables.
Ans: ARG = build time variable. ENV = runtime variable.

6. Which of the following is the process that is executed inside the docker container?
CMD, RUN, ENTRYPOINT
Ans: ENTRYPOINT.

7. How to create volumes and how do we declare in dockerfile?

1. Declaring a Volume in the Dockerfile
You can use the VOLUME instruction in the Dockerfile to define a mount point inside the container.

Example: Dockerfile with a Volume

```
# Use official Python image as base
FROM python:3.9
# Set the working directory inside the container
WORKDIR /app
# Copy the application files to the container
COPY . .
# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Define a volume to persist logs
```

```
VOLUME ["/app/logs"]
# Expose the application port
EXPOSE 5000
# Define the command to run the application
CMD ["python", "app.py"]
```

In this example, the `/app/logs` directory inside the container will be a volume.

---

2. Creating and Using a Named Volume
Instead of using the VOLUME instruction in the Dockerfile, you can explicitly create and attach a volume while running a container.
Create a Named Volume: `docker volume create mydata`
Run a Container and Attach the Volume: `docker run -d -p 5000:5000 --name my-container -v mydata:/app/logs my-python-app`
Here, `mydata` is the named volume that will persist data from the container's `/app/logs` directory.

---

3. Using Bind Mounts (Mount a Local Directory)
If you want to use a local directory on your host machine as a volume, use bind mounts:
`docker run -d -p 5000:5000 --name my-container -v $(pwd)/logs:/app/logs my-python-app`
This mounts the logs directory from your host machine to `/app/logs` inside the container.

---

4. Viewing and Managing Volumes
- **List all volumes:** `docker volume ls`
- **Inspect a volume:** `docker volume inspect mydata`
- **Remove a volume:** `docker volume rm mydata`

8. ENTRYPOINT and CMD and RUN difference.

Instruction	Purpose	Executed At	Example
RUN	Executes commands during the **image build** process	**Build time**	RUN apt-get update && apt-get install -y curl
CMD	Default command that runs when the container starts	**Runtime**	CMD ["python", "app.py"]
ENTRYPOINT	Defines a fixed command that always runs, even if arguments are provided	**Runtime**	ENTRYPOINT ["python", "app.py"]

---

1. RUN (Executed at Build Time)
- Used to execute commands **while building the image**.
- Typically used for installing dependencies, configuring the environment, etc.
- Creates a new image layer.

Example:
`RUN apt-get update && apt-get install -y curl`
- This installs curl during the image build process.
- This command **does not execute at runtime**.

---

2. CMD (Executed at Runtime)
- Specifies the default command for a container.
- Can be **overridden** when running the container.
- If multiple CMD instructions exist, only the **last one** takes effect.

Example:
`CMD ["python", "app.py"]`
- This means when you run `docker run my-image`, it will execute: `python app.py`
- You can override it:
  `docker run my-image python another_script.py`
  Now, `another_script.py` runs instead of `app.py`.

---

3. ENTRYPOINT (Executed at Runtime, Non-Overridable by Default)

- Used to specify a command that **always runs**.
- Unlike CMD, arguments passed to `docker run` are **appended** to the `ENTRYPOINT` command.
- If you want to allow overriding, use `ENTRYPOINT` with `CMD`.

Example 1: Fixed ENTRYPOINT

```
ENTRYPOINT ["python", "app.py"]
```

- Even if you run:
  `docker run my-image another_script.py`
  It **won't override** `ENTRYPOINT`, so it will still run `python app.py`.

Example 2: Flexible ENTRYPOINT

```
ENTRYPOINT ["python"]
CMD ["app.py"]
```

- Now, if you run:
  `docker run my-image`
  It runs:
  `python app.py`
- But if you run:
  `docker run my-image another_script.py`
  It runs:
  `python another_script.py`

Key Differences in Execution Order

If both ENTRYPOINT and CMD are defined:

1. ENTRYPOINT executes first.
2. CMD is passed as an argument to ENTRYPOINT.

For example:

```
ENTRYPOINT ["echo"]
CMD ["Hello, World!"]
```

Running `docker run my-image` outputs:
Hello, World!

When to Use What?

Use Case	Recommended Instruction
Installing packages or configuring the system	`RUN`
Default command but allow overrides	`CMD`
Ensuring a command always runs	`ENTRYPOINT`
Combining both (allowing arguments but having a default)	`ENTRYPOINT + CMD`

9. ADD and COPY difference.

Both ADD and COPY are used to copy files and directories into a Docker image, but they have some key differences.

Feature	COPY	ADD
Basic Purpose	Copies files/directories from host to container	Copies files/directories from host to container
Handles Remote URLs?	❌ No	❌ No
Extracts TAR Archives?	❌ No	✅ Yes (automatically extracts .tar, .tar.gz, .tar.bz2)
Supports Local Files & Directories?	✅ Yes	✅ Yes
Supports Remote URLs?	❌ No	❌ No
Preferred For?	Simple file copying	File copying + archive extraction

---

1. COPY - Basic File Copying

- Copies files and directories **as they are** from the host machine to the container.
- **Does not extract archives**.

Example: Using COPY

```
FROM ubuntu:latest
WORKDIR /app
# Copy local files into the container
COPY myfile.txt /app/
COPY myfolder /app/
```

📌 **Note:**

- myfile.txt is copied **without** any modifications.
- myfolder/ is copied **as a whole directory**.

---

2. ADD - File Copying with Extra Features

- Works like COPY, but **also** supports:
  - **Extracting .tar archives** automatically.
  - **Copying files from local sources** (not from URLs).

Example: Using ADD

```
FROM ubuntu:latest
WORKDIR /app
# Copy a regular file (same as COPY)
ADD myfile.txt /app/
# Copy and extract a .tar.gz archive automatically
ADD myarchive.tar.gz /app/
```

📌 **Key Difference:**

- If `myarchive.tar.gz` is added using `ADD`, it will **automatically be extracted** into `/app/`.

---

3. COPY vs ADD - When to Use?

Scenario	Use COPY	Use ADD
Copying individual files	✅ Yes	✅ Yes
Copying directories	✅ Yes	✅ Yes
Copying and **extracting** a .tar.gz file	❌ No	✅ Yes
Copying files from a URL	❌ No (use curl/wget)	❌ No (use curl/wget)

---

4. Why Prefer COPY Over ADD?

- **Security**: ADD extracts archives **without user control**, which can lead to unwanted behavior.
- **Explicit Control**: COPY only copies, while ADD has additional features that may be unnecessary.
- **Best Practice**: Use COPY unless you **specifically need** automatic .tar extraction.

---

5. How to Download from a URL (Since ADD Doesn't Support It)?

Instead of using ADD to download files, use RUN with curl or wget:

```
RUN curl -o /app/file.zip https://example.com/file.zip
RUN wget -O /app/file.zip https://example.com/file.zip
```

Final Best Practice:

✅ **Use COPY by default.**

✅ **Use ADD only when you need archive extraction.**

10. If containers dies we will lose the data what ull do for that?
11. How to create volumes? Why we need docker volumes?

When a container dies, any data stored inside it is **lost** unless we persist it using **volumes** or **bind mounts**. Here's how you can ensure your data is safe:

---

1. Use Docker Volumes (Recommended)

Docker **volumes** store data outside the container in Docker-managed storage, ensuring it persists even if the container stops or is deleted.

Steps to Use Volumes:

1. **Create a Volume**
   `docker volume create mydata`
2. **Run the Container with the Volume Attached**
   `docker run -d -p 5000:5000 --name my-container -v mydata:/app/data my-python-app`
   - This mounts `mydata` to `/app/data` inside the container.
   - If the container dies, the data is still in `mydata`.
3. **Verify Volume Data**
   `docker volume inspect mydata`

---

2. Use Bind Mounts (Host Storage)

If you want to store data on your **host machine's filesystem**, use a **bind mount**.

Steps to Use Bind Mounts:

```
docker run -d -p 5000:5000 --name my-container -v $(pwd)/data:/app/data my-python-app
```

- This mounts `./data` (a folder on the host) to `/app/data` in the container.
- Data will persist even if the container dies.

---

3. Use Stateful Storage for Databases

If you are running **databases** like MySQL or PostgreSQL, use a **named volume**.

Example for MySQL:

```
docker volume create mysql_data
docker run -d --name mysql-container -e MYSQL_ROOT_PASSWORD=root -v mysql_data:/var/lib/mysql mysql:latest
```

- MySQL data is stored in `mysql_data`, preventing data loss when the container stops.

---

4. Restart Policy to Handle Container Failures

To prevent the container from stopping permanently, use the --restart policy:

Restart Policy	Behavior
no (default)	Container doesn't restart if it dies
always	Container always restarts
on-failure	Restarts only if the container exits with an error
unless-stopped	Restarts unless manually stopped

Example:

```
docker run -d --restart=always -p 5000:5000 --name my-container -v mydata:/app/data my-python-app
```

- This ensures the container **restarts automatically** even after a crash.

---

5. Backup and Restore Data

Backup a Volume

```
docker run --rm -v mydata:/data -v $(pwd):/backup busybox tar -czf /backup/backup.tar.gz -C /data .
```

Restore a Volume

```
docker run --rm -v mydata:/data -v $(pwd):/backup busybox tar -xzf /backup/backup.tar.gz -C /data
```

Summary: Best Practices

✅ **Use Volumes** for persistent data.

✅ **Use Bind Mounts** if you want data on the host system.

✅ **Use Restart Policies** to automatically restart containers.

✅ **Backup and Restore Volumes** to prevent data loss.

12. What are the network type we have in docker? Why we require them can u explain?

Docker provides different network types to **enable communication** between containers, the host, and external networks.

---

1. Bridge Network (Default for Standalone Containers)

- **Command to create & use:**
```
docker network create my-bridge-network
docker run -d --name container1 --network my-bridge-network my-image
docker run -d --name container2 --network my-bridge-network my-image
```
- **How it works:**
  - Containers within the **same bridge network** can communicate using **container names**.
  - Containers in **different bridge networks** cannot communicate.
- **Use Case:**
  - Best for **isolated applications** where multiple containers need to talk to each other (e.g., **Web + Database**).

---

2. Host Network (Shares the Host's Network)

- **Command to run a container on the host network:**
```
docker run -d --network host --name my-container my-image
```
- **How it works:**
  - The container **shares** the host machine's network stack.
  - No separate **internal IP** for the container.
  - Ports are directly exposed on the **host machine**.
- **Use Case:**
  - When you want the container to communicate **as if it's running directly on the host**, such as high-performance applications.

---

3. None Network (Completely Isolated)

- **Command to run a container with no network:**
```
docker run -d --network none --name isolated-container my-image
```
- **How it works:**
  - The container **has no network access** (no internet, no communication with other containers).
- **Use Case:**
  - Best for **security-sensitive applications** where isolation is needed.

---

4. Overlay Network (For Multi-Host Swarm Communication)

- **Command to create & use overlay network (requires Docker Swarm):**
```
docker network create --driver overlay my-overlay-network
docker service create --name my-service --network my-overlay-network
my-image
```
- **How it works:**
  - Allows containers running on **different Docker hosts** to communicate.
  - Used in **Docker Swarm** for **multi-host deployments**.
- **Use Case:**
  - **Microservices architectures** across multiple machines.

---

5. Macvlan Network (Assigns Containers a Physical Network Address)

- **Command to create & use Macvlan network:**
```
docker network create -d macvlan \
    --subnet=192.168.1.0/24 \
    --gateway=192.168.1.1 \
    -o parent=eth0 my-macvlan-network
docker run -d --network my-macvlan-network --name my-container my-
image
```
- **How it works:**
  - Containers get **real IPs** on the physical network.
  - They appear as **separate devices** on the network.

- **Use Case:**
  - When you need containers to behave like **physical machines** (e.g., legacy systems).

Comparison Table

Network Type	Container-to-Container Communication	Can Access Host Network?	Use Case
Bridge (default)	✅ Yes (within the same bridge)	⊘ No	Best for containerized apps (e.g., web + DB)
Host	✅ Yes (same as host)	✅ Yes (directly shares host network)	High-performance apps needing direct host access
None	⊘ No	⊘ No	Completely isolated containers
Overlay	✅ Yes (across multiple Docker hosts)	✅ Yes	Multi-host services in Docker Swarm
Macvlan	✅ Yes (appears as a real device)	✅ Yes	When containers need real network identity

---

Which Network Should You Use?
✓ **For multi-container apps on a single host →** bridge
✓ **For best network performance (direct access) →** host
✓ **For full isolation →** none
✓ **For multi-host communication (Swarm) →** overlay
✓ **For containers acting like physical devices →** macvlan

13. How do you reduce size of the dockerfile?

Optimization	Why?
Use Alpine/Debian Slim	Smaller base image
Multi-stage builds	Removes unnecessary files
Combine RUN commands	Reduces layers
Use .dockerignore	Excludes unnecessary files
Remove unnecessary packages	Prevents bloat
Clean up after installing	Frees up space
Use COPY instead of ADD	Prevents unwanted extractions
Use distroless/scratch	Minimal runtime dependencies
Use slim or alpine images	Reduces base image size
Optimize caching order	Faster builds

14. Explain docker layer structure

When you build a Docker image, it's not just one big file. Instead, it's made up of **layers**, where **each instruction** in the Dockerfile (like FROM, RUN, COPY, etc.) creates a **new image layer**.
These layers are:

- **Stacked one on top of another**
- **Cached** for efficiency
- **Read-only**, except for the top container layer

📚 Types of Layers in Docker

Layer Type	Description
Base Image Layer	Created from FROM instruction. It could be ubuntu, alpine, etc.

Layer Type	Description
Intermediate Layers	Created by instructions like RUN, COPY, ADD, ENV, etc. Each creates a new read-only layer.
Top Layer (Container Layer)	When you run a container, Docker adds a **writable layer** on top where your container can make changes.

🏗 Example: Dockerfile Layer Breakdown

```
FROM python:3.9-slim          # Layer 1: Base image
WORKDIR /app                  # Layer 2: Set working directory
COPY requirements.txt .       # Layer 3: Copy requirements file
RUN pip install -r requirements.txt  # Layer 4: Install packages
COPY . .                      # Layer 5: Copy app code
CMD ["python", "app.py"]      # Layer 6: Default command
```

Each line becomes a **layer** in the final image. When you rebuild the image:

- Docker **reuses cached layers** if the instructions haven't changed.
- Only the changed or new layers are rebuilt.

💡 Key Benefits of Docker Layers

Feature	Benefit
Layer Caching	Faster builds because unchanged layers are reused.
Image Reuse	Common base layers can be shared across multiple images.
Efficient Storage	Only unique layers are stored; duplicates are avoided.
Version Control-like	You can roll back to earlier images using the layer history.

🔍 Inspecting Layers

Use the following command: `docker history <image_name>`

✅ Example: `docker history myapp:latest`

This shows all layers with command history, sizes, and timestamps.

⚠ Layering Best Practices

Tip	Why?
Combine RUN commands	Reduce number of layers and image size.
Use .dockerignore	Avoid unnecessary files getting copied into layers.
Keep COPY and ADD at the bottom	They change frequently, so reduce cache busting.
Use small base images (alpine)	Keep image sizes minimal.

📌 Final Notes

- All layers **below the container layer** are **read-only**.
- Containers write changes to the **top writable layer** (this is ephemeral and lost if the container is deleted unless you use volumes).

15. Docker architecture. (Refer Sriman sir notes)
16. Write few docker cmds u have used
17. How do you list all docker containers?

1. Container Management Commands

Command	Description	Example
`docker run`	Runs a new container from an image	`docker run -d --name mycontainer nginx`
`docker ps`	Lists running containers	`docker ps`

Command	Description	Example
`docker ps -a`	Lists all containers (including stopped ones)	`docker ps -a`
`docker stop`	Stops a running container	`docker stop mycontainer`
`docker start`	Starts a stopped container	`docker start mycontainer`
`docker restart`	Restarts a container	`docker restart mycontainer`
`docker rm`	Removes a stopped container	`docker rm mycontainer`
`docker logs`	Displays logs of a container	`docker logs -f mycontainer`
`docker exec`	Runs a command inside a running container	`docker exec -it mycontainer bash`
`docker inspect`	Shows detailed information about a container	`docker inspect mycontainer`

---

2. Image Management Commands

Command	Description	Example
`docker images`	Lists all available images	`docker images`
`docker pull`	Downloads an image from Docker Hub	`docker pull nginx:latest`
`docker build`	Builds an image from a Dockerfile	`docker build -t myapp .`
`docker tag`	Tags an image with a new name	`docker tag myapp myrepo/myapp:v1`
`docker rmi`	Removes an image	`docker rmi nginx`
`docker history`	Shows layers of an image	`docker history nginx`

---

3. Volume Management Commands

Command	Description	Example
`docker volume create`	Creates a new volume	`docker volume create myvolume`
`docker volume ls`	Lists all volumes	`docker volume ls`
`docker volume rm`	Removes a volume	`docker volume rm myvolume`
`docker run -v`	Mounts a volume to a container	`docker run -d -v myvolume:/data nginx`
`docker inspect volume`	Shows details of a volume	`docker inspect myvolume`

---

4. Network Management Commands

Command	Description	Example
`docker network ls`	Lists all networks	`docker network ls`
`docker network create`	Creates a new network	`docker network create mynetwork`
`docker network inspect`	Shows details of a network	`docker network inspect mynetwork`
`docker network connect`	Connects a container to a network	`docker network connect mynetwork mycontainer`

Command	Description	Example
`docker network disconnect`	Disconnects a container from a network	`docker network disconnect mynetwork mycontainer`
`docker run --network`	Runs a container in a specific network	`docker run -d --network mynetwork nginx`

---

5. Docker Compose Commands

Command	Description	Example
`docker-compose up`	Starts all services defined in `docker-compose.yml`	`docker-compose up -d`
`docker-compose down`	Stops and removes all containers	`docker-compose down`
`docker-compose ps`	Lists running containers in a Compose project	`docker-compose ps`
`docker-compose logs`	Displays logs for all services	`docker-compose logs -f`
`docker-compose exec`	Runs a command inside a running service container	`docker-compose exec web bash`

---

6. Docker Registry Commands

Command	Description	Example
`docker login`	Logs in to a Docker registry	`docker login -u myusername`
`docker logout`	Logs out from a Docker registry	`docker logout`
`docker push`	Uploads an image to a registry	`docker push myrepo/myapp:v1`
`docker pull`	Downloads an image from a registry	`docker pull myrepo/myapp:v1`

---

7. Cleanup Commands

Command	Description	Example
`docker system prune`	Removes unused containers, networks, and images	`docker system prune -a`
`docker container prune`	Removes all stopped containers	`docker container prune`
`docker image prune`	Removes unused images	`docker image prune -a`
`docker volume prune`	Removes unused volumes	`docker volume prune`

Summary

- `docker run` → Start a container
- `docker ps` → List running containers
- `docker stop` → Stop a container
- `docker rm` → Remove a container
- `docker images` → List images
- `docker rmi` → Remove an image
- `docker volume` → Manage persistent data
- `docker network` → Configure networking
- `docker-compose` → Manage multi-container apps
- `docker system prune` → Clean up unused resources

18. How to ensure your using secure docker image?

1. Use Official and Verified Images

✅ **Always pull images from trusted sources**, such as:

- **Docker Official Images** → Maintained by Docker & verified (e.g., nginx:latest)
- **Verified Publishers** → Vendors like Microsoft, AWS, etc.
- **Private Registry** → Use a secure internal repository

🔹 **Example (Good Practice):**

```
docker pull nginx:latest   # Pulling an official image
```

🚫 **Avoid:**

```
docker pull random-user/nginx   # Unverified third-party image
```

---

2. Use Minimal Base Images (Alpine, Distroless, Scratch)

Using **lightweight** base images reduces attack surfaces.

✔ **Recommended Base Images:**

- alpine → ~5MB, minimal attack surface
- distroless → No package manager, highly secure
- scratch → Empty base (for Go, Rust apps)

🔹 **Example (Good Practice):**

```
FROM alpine:latest   # Secure and minimal base image
```

🚫 **Avoid:**

```
FROM ubuntu:latest   # Larger image, more attack surface
```

---

3. Scan Images for Vulnerabilities

Use **security scanning tools** to detect vulnerabilities in images.

🔍 **Recommended Scanners:**

Tool	Usage
Trivy	trivy image myapp:latest
Docker Scout	docker scout quickview myapp:latest
Grype	grype myapp:latest
Snyk	snyk container test myapp:latest

🔹 **Example: Trivy Scan**

```
trivy image myapp:latest
```

✅ **Fix vulnerabilities before deployment**.

---

4. Avoid Using latest Tag

- The latest tag is **not always the most secure version**.
- Always **pin versions** to a **specific tag** (e.g., 1.21.4 instead of latest).

🔹 **Example (Good Practice)**

```
docker pull nginx:1.21.4
```

🚫 **Avoid:**

```
docker pull nginx:latest
```

---

5. Use Multi-Stage Builds to Reduce Attack Surface

- Multi-stage builds remove unnecessary dependencies from the final image.

🔹 **Example (Good Practice)**

```
# Build stage
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

# Final stage (secure and minimal)
FROM alpine:latest
WORKDIR /app
```

```
COPY --from=builder /app/myapp .
CMD ["./myapp"]
```

✅ **Reduces image size and removes build tools**.

---

6. Remove Unnecessary Packages and Tools

- Avoid installing extra tools (like curl, wget, vim) in production images.

🔹 **Example (Good Practice)**

```
RUN apk add --no-cache bash
```

🔸 **Avoid:**

```
RUN apt-get install -y curl wget vim   # Adds unnecessary packages
```

---

7. Set Proper User Permissions (Avoid Root User)

- Running a container as **root** is a security risk.
- Always **create a non-root user** and run the app as that user.

🔹 **Example (Good Practice)**

```
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
CMD ["node", "server.js"]
```

🔸 **Avoid:**

```
USER root  # Security risk!
```

---

8. Enable Content Trust (DOCKER_CONTENT_TRUST=1)

This ensures you pull **signed and verified** images.

🔹 **Enable it globally:**

```
export DOCKER_CONTENT_TRUST=1
docker pull nginx:latest  # Only pulls signed images
```

---

9. Restrict Privileged Mode & Capabilities

- Containers **should not run in privileged mode** unless necessary.
- Restrict Linux capabilities using --cap-drop ALL.

🔹 **Example (Good Practice)**

```
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE myapp
```

🔸 **Avoid:**

```
docker run --privileged myapp  # Security risk!
```

---

10. Regularly Update and Rebuild Images

- Keep your images **updated** to receive security patches.
- Automate updates using **CI/CD pipelines**.

🔹 **Example: Rebuild Image**

```
docker build -t myapp:1.0 .
docker push myapp:1.0
```

🛡**Summary: Docker Image Security Best Practices**

Best Practice	Why?
✅ Use **official and verified images**	Avoid using untrusted sources
✅ Use **minimal base images** (alpine, distroless)	Reduce attack surface
✅ **Scan images** for vulnerabilities (Trivy, Grype, Snyk)	Detect security flaws
✅ **Pin image versions** instead of using latest	Ensure stability & security
✅ **Use multi-stage builds**	Reduce image size and remove build tools
✅ **Drop unnecessary privileges** (avoid root user)	Minimize security risks
✅ **Enable Docker Content Trust**	Pull only signed images

Best Practice	Why?
✅ **Limit container capabilities** (--cap-drop ALL)	Restrict unnecessary permissions
✅ **Regularly update and rebuild images**	Patch vulnerabilities

19. What is docker namespace? Y we need namespace?

A **Docker namespace** is a feature of the Linux kernel that **isolates processes** and provides **independent execution environments** for containers. It ensures that containers running on the same host **don't interfere** with each other by providing separate instances of resources like **process IDs, network interfaces, mount points, and more**.

◆ **Why Do We Need Namespaces?**

Namespaces are required in Docker to **achieve process isolation**, ensuring that containers:

1. **Operate independently** (without affecting other containers or the host system).
2. **Have their own isolated network, process tree, and filesystems**.
3. **Improve security** by restricting access to system resources.

Types of Namespaces in Docker

Docker uses multiple Linux namespaces for container isolation:

Namespace	Purpose	Example
PID Namespace (pid)	Isolates process IDs	Each container has its own PID 1, so processes inside one container **cannot see or kill** processes in another container.
Network Namespace (net)	Provides isolated networking	Each container gets a separate network stack with its own IP address, routes, and ports.
Mount Namespace (mnt)	Isolates the filesystem	Each container has its own file system and cannot see or modify host or other container files.
UTS Namespace (uts)	Isolates host and domain names	Containers can have different hostname and domainname than the host system.
IPC Namespace (ipc)	Isolates shared memory and signals	Containers cannot access shared memory (e.g., semaphores, message queues) of other containers or the host.
User Namespace (user)	Provides user isolation	A container can run as a **non-root user** while mapping to root inside the container.

◆ **Summary: Why Use Docker Namespaces?**

✅ **Process Isolation** → Prevents containers from interfering with each other.

✅ **Security** → Restricts access to system resources.

✅ **Resource Management** → Each container gets its own network, file system, and process tree.

✅ **Scalability** → Allows running multiple containers without conflicts.

20. Explain multistage dockerfile.

A **Multi-Stage Dockerfile** is a technique to **build lightweight Docker images** by using **multiple stages**. It helps in:

✅ **Reducing image size** by removing unnecessary build dependencies.

✅ **Improving security** by minimizing attack surfaces.

✅ **Optimizing performance** by keeping only the required files.

How Does Multi-Stage Build Work?

- The **first stage** (builder) compiles the application.
- The **final stage** copies only the required files to a minimal image.
- Intermediate layers are **discarded**, reducing the image size.

Example 1: Multi-Stage Build for a Golang App

Before Multi-Stage (Large Image)

```
FROM golang:1.21
WORKDIR /app
COPY . .
RUN go build -o myapp
CMD ["./myapp"]
```

🚨 **Problem:** This image includes the entire **Go compiler**, which is **not needed** at runtime.

After Multi-Stage (Optimized Image)

```
# Stage 1: Build the application
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .
RUN go build -o myapp

# Stage 2: Create a lightweight final image
FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/myapp .
CMD ["./myapp"]
```

✅ **Result:**

- The final image **only contains the compiled binary** (myapp).
- The **Go compiler is removed**, reducing the image size **from ~800MB to ~10MB**.

Example 2: Multi-Stage Build for a Node.js App

```
# Stage 1: Install dependencies and build
FROM node:18 AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Create a minimal production image
FROM node:18-alpine
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
CMD ["node", "dist/server.js"]
```

✅ **Why Multi-Stage?**

- **Stage 1:** Installs dependencies and builds the app.
- **Stage 2:** Copies only the required files (dist/, node_modules/).
- **Final Image is much smaller** because npm install temporary files are **removed**.

Example 3: Multi-Stage Build for a Java Application

```
# Stage 1: Build the JAR file
FROM maven:3.9 AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn package -DskipTests

# Stage 2: Run the JAR file in a lightweight image
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY --from=builder /app/target/myapp.jar myapp.jar
CMD ["java", "-jar", "myapp.jar"]
```

✅ **Why Multi-Stage?**

- **Maven dependencies** are removed from the final image.
- The **OpenJDK slim version** is used instead of a full JDK.

When Should You Use Multi-Stage Builds?

✓ **Compiling apps (Go, Java, C++, Node.js, etc.)**

✓ **Reducing image size by discarding unnecessary files**

✓ **Minimizing dependencies in the final image**

✓ **Improving security by removing build tools**

Summary

Traditional Build ✗	**Multi-Stage Build ✓**
Large images with unnecessary files	**Minimal images** with only required files
Security risks (contains compilers & tools)	**More secure** (no extra tools)
Slow deployments	**Faster deployments**
No caching optimization	**Uses Docker caching for efficiency**

21. Explain how you create a docker image.
22. ECR. Have you used and what is it?
23. Is docker registry onpremise or on cloud?
24. How ull authenticate to ecs
25. How do you push to docker hub or ecr?
26. Which changes should you make in a personal Docker Compose file before it moves to production?
27. Explain Docker Swarm very briefly?
28. Your friend/colleague has tried to create a container and it failed/not working. He's asking your help to troubleshoot but he hasn't shared credentials to docker cli. How do you get the container id and how do you inspect/troubleshoot without having access to docker?
29. If my container is not working how do you troubleshoot it?
30. The prod version docker image had some issues how will u revert the changes
31. What types of docker files u have written & build pipeline? Like java project or even other language applications?

Kubernetes: Pod, Service, ConfigMap, Secret, PersistentVolume, Namespace, Deployment, StatefulSet, DaemonSet, ReplicaSet

1. Deployment stratergies in K8s. What is blue green deployment?
2. What type of deployment strategy you have applied in production?
3. How ull rollback deployments in k8s? If there is any issue in the deployment how ull rollback? How do you rollback to previous step in kubernetes incase pod is failed?
4. How do you setup and work with Kubernetes?
5. Write a deployment file in K8s.
6. Write a service file in K8s.
7. What are services & types?
8. Can you write any one service and deployment manifest files?
9. Difference between stateful sets and deployment sets?
10. How ull dryrun the manifest files in k8s
11. What is manifest file?
12. What is headless services in Kubernetes?
13. Have you worked with statefulsets and stateful applications?
14. Difference b/w replica set & replica controller
15. What is daemon set & y we need them
16. Have u worked on HPA..just tel me brief overview of it
17. Differnece between ClusterIP, Nodeport and Loadbalancers.
18. What is ConfigMaps and Secrets and what do we store it? Can anyone access the data in ConfigSecrets?
19. Have you created any AWS Secrets and integrated with K8s?
20. What is ingress and why do you use?
21. Pick the invalid service type in Kubernetes from the list.

Cluster IP, Node port, ingress.
Ans: Ingress.
22. How do you secure cluster?
23. How do you give internal cluster access to your pod?
24. How do you expose your kubernetes pod to external world?
25. How do you ensure your pod communicates with the cluster?
26. Explain K8s architecture. How ull secure k8 architecture?
27. What is scheduler & api gateway
28. What is kubernetes controller manager?
29. What is minikube?
30. Y we need resource requests in k8s
31. Y we need liveness probe & readiness probe
32. How to ensure that only one instance of pod is running on every node of a kubernetes cluster?
You call this as a daemon set, replica set or stateful set.
Ans: Daemon set.
33. How do you deploy the pods in k8s what is the procedure
34. What is difference between pod and node?
35. What is namespace in K8s? Why do we use it?
36. Cmd to list all nodes
37. Cmd to list the nodes of perticular namespace
38. Will It list all pods if I type kubectl get pods?
39. kubectl get logs command is used to get the log of events in kubernetes cluster. True or false.
Ans: True.
40. How check the resource how much the pod is consuming/ allocated
41. How to attach volumes in docker & k8s
42. What is pv & pvc. How ull request & whats the process to mount it
43. How do you use persistent volume in kubernetes?
44. EKS. How to create and login to EKS cluster?
45. How do you troubleshoot the pod if it suddenly stops?
46. If we declare like replicas=2 then if we want to scale up how do we do it? We need to do it automatically? (Hint: Horizontal scaling will happen automatically)
47. If pod dies what ull do..where & what ull check
48. How do you upgrade node in production?
49. If pod dies, how you gonna check logs?
50. How will you scale up the pods automatically? & Cmd to scale up the replicas manually. Kubectl cmd to increase the replicas.
51. Which version of Kubernetes do you use?
52. Recent issue faced in Kubernetes and how do you fix it? What are the error that you faced in k8s?
53. Who will handle the crashed pod scenarios?
54. The request is in transit the pod will go to unresponsive/crashed then how ull handle these kind of Transit request which are going to fail
55. I want perticular requests to be routed to perticular pod based on location how do u handle the traffic
56. I have an image image:web:3.0, app: my-web-app and namespace is hexaware. I want to deploy this image in my namespace hexaware. How do you do? (Hint: We can do this without writing deployment manifest as well by inline)
57. How do you troubleshoot pod in kubernetes if it's not starting and how do you fix it?

Jenkins:
Pipeline:
1. What all you have done using Jenkins?
2. Write any Jenkins Pipeline. In pipeline what is agent? Why do we need agent? Explain Jenkins pipeline. What all agents have you used in pipeline? Have you used any cloud agents?
3. In Jenkins pipeline we mentioned maven and jdk under tools. So they asked we have 2 projects A and B. A and B need different versions of jdk. How do we achieve this?

4. How to compile java and C# code in CI CD pipelines?
5. What tool u use for cicd...What type of Jenkins pipeline do u build?
6. Pick the valid job types in Jenkins from the following. There might be 2 correct answers if you guess even one of 2 that is fine.
Multi Configuration project, Multi module Project, Organization Folder, Multi branch pipeline.
Ans: Multi Branch is there.
7. What kind of pipeline you have written and implemented? Explain the steps.
8. Why we need to use multi-branch pipeline? When & how to construct that?
9. Difference b/w declarative & structured pipeline. Write a declarative pipeline & explain the entire flow
10. CD integration
11. What are different types of pipelines in Jenkins?
12. Which construct is used to execute a pipeline stage only if a specific condition is met?
When directive, if else condition, test command.
Ans: if else condition.
Plugins and Integration:
13. How can you create a backup and copy files in Jenkins?
14. Do u know to take a backup of Jenkins?
15. What could be the steps to move or copy Jenkins from one server to another?
16. Which component is responsible for executing task within Jenkins job?
Jenkins built agent, Jenkins built executor, Jenkins task executor.
Ans: Jenkins built agent.
17. Name some of the useful plugins in Jenkins?
18. What are the security measures you have applied to those pipelines?
19. How do you going to integrate k8s with Jenkins?
20. You do not have any connection between AWS and Jenkins. How do you connect and integrate with Jenkins?
21. How do you integrate static code test in your Jenkins pipeline?
22. Have u created any environmental variables in Jenkins
23. Explain blue green deployment in CD part of CI CD pipeline.
Scenario and Troubleshooting:
24. How do you get to know if there are issues in Jenkins build? How do you troubleshoot?
25. After the code commit Jenkins was not starting how do you troubleshoot?
26. Let's say there is a broken build in the Jenkins project, then what can be done?
27. I have a Jenkins single pipeline and a single branch and I want to use single branch and single pipeline and execute the same on multiple environments how do we do it?
28. How can you parallelly trigger multiple Jenkins builds to speed up the process?